Implement Hill Climbing search algorithm to solve N-Queens problem

```
def cost(state):
    h = 0

    for i in range(4):
        for j in range(i + 1, 4):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                h += 1

    return h


def print_board(state):
    for i in range(4):
        for j in range(4):
            if state[j] == i:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
    print()


def hill_climbing():
    current = [3, 1, 2, 0]

    print("Initial State:", current)
    print("Cost:", cost(current))
    print_board(current)

    while cost(current) != 0:
        neighbors = []

        for i in range(4):
            for j in range(i + 1, 4):
                neighbor = current.copy()
                neighbor[i], neighbor[j] = neighbor[j], neighbor[i]
                neighbors.append((cost(neighbor), i, j, neighbor))

        best = min(neighbors)

        if best[0] >= cost(current):
            print("Local optimum reached.")
            break

        current = best[3]

        print("Current State:", current)
```

```
        print("Cost:", cost(current))
        print_board(current)

    if cost(current) == 0:
        print("Goal State:", current)
        print("Final Cost:", cost(current))


hill_climbing()
```

```
rs/BMSCE/Desktop/1BM24CS327/4queen.py
Initial State: [3, 1, 2, 0]
Cost: 2
. . . Q
. Q . .
. . Q .
Q . . .

Current State: [1, 3, 2, 0]
Cost: 1
. . . Q
Q . . .
. . Q .
. Q . .

Current State: [1, 3, 0, 2]
Cost: 0
. . Q .
Q . . .
. . . Q
. Q . .

Goal State: [1, 3, 0, 2]
Final Cost: 0
```